

```
import pandas as pd

batting_stats = pd.read_csv("batting_stats.csv")
bowling_stats = pd.read_csv("bowling_stats.csv")
match_records = pd.read_csv("match_records.csv")
```

```
import pandas as pd
import numpy as np
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.impute import SimpleImputer
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline

# --- Reload dataframes to ensure a clean state ---
# This prevents issues if dataframes were modified in-place by previous cell
batting_stats = pd.read_csv("batting_stats.csv")
bowling_stats = pd.read_csv("bowling_stats.csv")
match_records = pd.read_csv("match_records.csv")

# --- 1. Handling Missing Values, Removing Duplicates, and Feature Engineeri

### Preprocessing batting_stats DataFrame ###
print("Preprocessing batting_stats...")

# 1. Removing duplicates
batting_stats.drop_duplicates(inplace=True)
print(f"Batting stats after removing duplicates: {batting_stats.shape[0]} ro

# 2. Feature Engineering for batting_stats
# Calculate Total Boundaries
batting_stats['Total_Boundaries'] = batting_stats['Fours'] + batting_stats['

# Calculate Boundary Percentage of Runs (handle division by zero)
batting_stats['Boundary_Percentage_of_Runs'] = np.where(
    batting_stats['Runs'] > 0,
    (batting_stats['Fours'] * 4 + batting_stats['Sixes'] * 6) / batting_stat
    0
)

# Calculate Big Scores Ratio (Hundreds + Fifties per Innings, handle divisio
batting_stats['Big_Scores_Ratio'] = np.where(
    batting_stats['Innings'] > 0,
    (batting_stats['Hundreds'] + batting_stats['Fifties']) / batting_stats['
    0
)

# Define numerical and categorical features for batting_stats
# Corrected 'Balls' to 'Balls_faced' and 'Strike_Rate' to 'Strike_rate' base
batting_numerical_features = [
    'Position', 'Matches', 'Innings', 'Runs', 'Average', 'Balls_faced', 'Str
    'Fours', 'Sixes', 'Hundreds', 'Fifties', 'Ducks',
    'Total_Boundaries', 'Boundary_Percentage_of_Runs', 'Big_Scores_Ratio'
```

```

]
batting_categorical_features = ['Country']

# Create preprocessing pipelines for numerical and categorical features
batting_numerical_transformer = Pipeline(steps=[
    ('imputer', SimpleImputer(strategy='median')), # Impute missing numerical
    ('scaler', StandardScaler()) # Scale numerical features
])

batting_categorical_transformer = Pipeline(steps=[
    ('imputer', SimpleImputer(strategy='most_frequent')), # Impute missing categorical
    ('onehot', OneHotEncoder(handle_unknown='ignore')) # One-hot encode categorical
])

# Combine transformers using ColumnTransformer
batting_preprocessor = ColumnTransformer(
    transformers=[
        ('num', batting_numerical_transformer, batting_numerical_features),
        ('cat', batting_categorical_transformer, batting_categorical_features)
    ],
    remainder='passthrough' # Keep columns not specified (like 'Player', 'No
)

# Apply preprocessing to batting_stats
batting_processed = batting_preprocessor.fit_transform(batting_stats)

# Get feature names after one-hot encoding
batting_ohe_feature_names = batting_preprocessor.named_transformers_['cat'].

# Get the names of original columns that are not transformed (i.e., passed through)
original_batting_cols = batting_stats.columns.tolist()
passthrough_features_final_batting = [
    col for col in original_batting_cols
    if col not in batting_numerical_features and col not in batting_categorical_features
]

batting_all_features_ordered = (
    batting_numerical_features +
    list(batting_ohe_feature_names) +
    passthrough_features_final_batting
)

# Convert the processed data back to a DataFrame
batting_stats_processed_df = pd.DataFrame(batting_processed, columns=batting_all_features_ordered)
print("Batting stats preprocessing complete.")

### Preprocessing bowling_stats DataFrame ###
print("\nPreprocessing bowling_stats...")

# 1. Removing duplicates
bowling_stats.drop_duplicates(inplace=True)
print(f"Bowling stats after removing duplicates: {bowling_stats.shape[0]} rows")

# Custom function to convert Overs (e.g., '10.2' or '10') to total balls

```

```

def overs_to_balls(overs_val):
    try:
        s_overs = str(overs_val)
        parts = s_overs.split('.')
        full_overs = float(parts[0])
        balls = float(parts[1]) if len(parts) > 1 and parts[1].isdigit() else 0
        return (full_overs * 6) + balls
    except ValueError:
        return np.nan # Handle potential conversion errors

# 2. Feature Engineering for bowling_stats
# Calculate Total Wicket Hauls using actual column names '4WH' and '5WH'
bowling_stats['Total_Hauls'] = bowling_stats['4WH'] + bowling_stats['5WH']

# Calculate Wickets per Match (handle division by zero)
bowling_stats['Wickets_per_Match'] = np.where(
    bowling_stats['Matches'] > 0,
    bowling_stats['Wickets'] / bowling_stats['Matches'],
    0
)

# Calculate Overs_in_balls using the robust function
bowling_stats['Overs_in_balls'] = bowling_stats['Overs'].apply(overs_to_ball)

bowling_stats['Wickets_per_Ball'] = np.where(
    bowling_stats['Overs_in_balls'] > 0,
    bowling_stats['Wickets'] / bowling_stats['Overs_in_balls'],
    0
)

# Define numerical and categorical features for bowling_stats
# Removed 'Runs_Conceded' and 'Average', added '4WH', '5WH', 'Total_Hauls'
bowling_numerical_features = [
    'Position', 'Matches', 'Innings', 'Wickets', 'Overs',
    'Economy', '4WH', '5WH', 'Total_Hauls',
    'Wickets_per_Match', 'Wickets_per_Ball', 'Overs_in_balls'
]
bowling_categorical_features = ['Country']

# Create preprocessing pipelines for numerical and categorical features
bowling_numerical_transformer = Pipeline(steps=[
    ('imputer', SimpleImputer(strategy='median')), # Impute missing numerical
    ('scaler', StandardScaler()) # Scale numerical features
])

bowling_categorical_transformer = Pipeline(steps=[
    ('imputer', SimpleImputer(strategy='most_frequent')), # Impute missing categorical
    ('onehot', OneHotEncoder(handle_unknown='ignore')) # One-hot encode categorical
])

# Combine transformers using ColumnTransformer
bowling_preprocessor = ColumnTransformer(
    transformers=[
        ('num', bowling_numerical_transformer, bowling_numerical_features),
        ('cat', bowling_categorical_transformer, bowling_categorical_features)
    ]
)

```

```

    ],
    remainder='passthrough' # Keep columns not specified (like 'Player') as
)

# Apply preprocessing to bowling_stats
bowling_processed = bowling_preprocessor.fit_transform(bowling_stats)

# Get feature names after one-hot encoding
bowling_ohe_feature_names = bowling_preprocessor.named_transformers_['cat'].

original_bowling_cols = bowling_stats.columns.tolist()
passthrough_features_final_bowling = [
    col for col in original_bowling_cols
    if col not in bowling_numerical_features and col not in bowling_categori
]

bowling_all_features_ordered = (
    bowling_numerical_features +
    list(bowling_ohe_feature_names) +
    passthrough_features_final_bowling
)

# Convert the processed data back to a DataFrame
bowling_stats_processed_df = pd.DataFrame(bowling_processed, columns=bowling
print("Bowling stats preprocessing complete.")

### Preprocessing match_records DataFrame ###
print("\nPreprocessing match_records...")

# 1. Removing duplicates
match_records.drop_duplicates(inplace=True)
print(f"Match records after removing duplicates: {match_records.shape[0]} ro

# Explicitly convert numerical columns to numeric, coercing errors to NaN
# This addresses initial ValueError for 'object' dtypes before ColumnTransfo
match_numerical_features_to_convert_pre_ct = [
    '1st_score', '1st_wkts', '2nd_score', '2nd_wkts',
    'wb_runs', 'wb_wickets'
]

for col in match_numerical_features_to_convert_pre_ct:
    match_records[col] = pd.to_numeric(match_records[col], errors='coerce')

# 2. Feature Engineering for match_records
# Convert 'date' to datetime and extract features
match_records['date'] = pd.to_datetime(match_records['date'], format='%d-%m-
match_records['match_year'] = match_records['date'].dt.year
match_records['match_month'] = match_records['date'].dt.month
match_records['match_day_of_week'] = match_records['date'].dt.dayofweek

# The columns for overs faced by each team are not available in the dataset.
# Therefore, Run_Rate_Team1 and Run_Rate_Team2 cannot be calculated as previ
# We will proceed without these features for now.

```

```

# Binary feature: if team1 won the toss
match_records['Team1_Won_Toss'] = (match_records['toss_winner'] == match_rec

# Binary feature: if toss winner chose to bat
match_records['Toss_Winner_Batting'] = (match_records['toss_decision'] == 'B

# Binary feature: Home Advantage for Team 1 (assuming venue_country is home
match_records['Home_Advantage_Team1'] = (match_records['team1'] == match_rec

# Binary feature: Home Advantage for Team 2
match_records['Home_Advantage_Team2'] = (match_records['team2'] == match_rec

# Define numerical and categorical features for match_records
# Updated based on available columns from match_records.columns output.
# Moved 'DLS_involved' to categorical features as it contains 'Yes'/'No' val
match_numerical_features = [
    '1st_score', '1st_wkts', '2nd_score', '2nd_wkts',
    'wb_runs', 'wb_wickets',
    'match_year', 'match_month', 'match_day_of_week',
    'Team1_Won_Toss', 'Toss_Winner_Batting',
    'Home_Advantage_Team1', 'Home_Advantage_Team2'
]
match_categorical_features = [
    'team1', 'team2', 'match_type', 'venue_stadium', 'venue_city',
    'venue_country', 'toss_winner', 'toss_decision', 'result', 'winning_team'
]

# Create preprocessing pipelines for numerical and categorical features
match_numerical_transformer = Pipeline(steps=[
    ('imputer', SimpleImputer(strategy='median')), # Impute missing numerica
    ('scaler', StandardScaler()) # Scale numerical features
])

match_categorical_transformer = Pipeline(steps=[
    ('imputer', SimpleImputer(strategy='most_frequent')), # Impute missing c
    ('onehot', OneHotEncoder(handle_unknown='ignore')) # One-hot encode cate
])

# Combine transformers using ColumnTransformer
match_preprocessor = ColumnTransformer(
    transformers=[
        ('num', match_numerical_transformer, match_numerical_features),
        ('cat', match_categorical_transformer, match_categorical_features)
    ],
    remainder='passthrough' # Keep other columns not specified (like 'match_
)

# Apply preprocessing to match_records
# Drop original 'date' column as its components are extracted
match_processed = match_preprocessor.fit_transform(match_records.drop(column

# Get feature names after one-hot encoding
match_ohe_feature_names = match_preprocessor.named_transformers_['cat'].name

# Determine the actual passthrough columns after dropping 'date'

```

```
original_match_cols_after_date_drop = match_records.drop(columns=['date']).c
passthrough_features_final_match = [
    col for col in original_match_cols_after_date_drop
    if col not in match_numerical_features and col not in match_categorical_
]

# Construct the final list of column names for the processed DataFrame
match_all_features_ordered = (
    match_numerical_features +
    list(match_ohe_feature_names) +
    passthrough_features_final_match
)

# Convert the processed data back to a DataFrame
match_records_processed_df = pd.DataFrame(match_processed, columns=match_all

# Explicitly convert columns that should be numerical to float, coercing err
# This addresses the ValueError encountered by XGBoost due to 'object' dtype
columns_to_convert_to_numeric = [
    col for col in match_records_processed_df.columns
    if col not in passthrough_features_final_match # Don't convert the passt
]

for col in columns_to_convert_to_numeric:
    match_records_processed_df[col] = pd.to_numeric(match_records_processed_

# Impute any NaNs that might have been introduced by the above coerce (e.g.
# This is a safeguard, as SimpleImputer should have handled most NaNs from o
for col in columns_to_convert_to_numeric:
    if match_records_processed_df[col].isnull().any():
        median_val = match_records_processed_df[col].median()
        if pd.isna(median_val): # If median is still NaN (e.g., all values w
            match_records_processed_df[col].fillna(0, inplace=True) # Fallba
        else:
            match_records_processed_df[col].fillna(median_val, inplace=True)

print("Match records preprocessing complete.")

print("\n--- Preprocessing Summary ---")
print("Processed batting_stats_processed_df head:")
print(batting_stats_processed_df.head())
print("\nProcessed bowling_stats_processed_df head:")
print(bowling_stats_processed_df.head())
print("\nProcessed match_records_processed_df head:")
print(match_records_processed_df.head())
```

	Player	BBI	Balls	Maidens	Runs
0	Deepti Sharma	5--39	488	3	449
1	Annabel Sutherland	5--40	362	4	269
2	Sophie Ecclestone	4--17	337	9	228
3	Shree Charani	3--41	468	3	387
4	Alana King	7--18	336	8	226

[5 rows x 23 columns]

Processed match_records_processed_df head:

	1st_score	1st_wkts	2nd_score	2nd_wkts	wb_runs	wb_wickets	match_year
0	0.471002	-0.557370	0.232571	1.148954	-0.934565	-0.077703	0
1	1.300774	0.771744	0.638312	1.148954	0.186190	-0.077703	0
2	-1.567034	0.771744	-1.015863	-1.117619	0.167511	0.524494	0
3	-2.440477	0.771744	-1.920978	-2.089008	0.167511	2.331086	0
4	0.107067	0.107187	-0.017116	0.177566	0.167511	-0.077703	0

	match_month	match_day_of_week	Team1_Won_Toss	...	\
0	-3.937004	-1.148004	-0.849837	...	
1	0.000000	-0.639602	1.176697	...	
2	0.000000	-0.131200	1.176697	...	
3	0.000000	0.377201	-0.849837	...	
4	0.000000	0.885603	-0.849837	...	

	winning_team_South Africa	winning_team_Sri Lanka	DLS_involved_No	\
0	0.0	0.0	0.0	
1	0.0	0.0	1.0	
2	0.0	0.0	1.0	
3	0.0	0.0	1.0	
4	0.0	0.0	1.0	

	DLS_involved_Yes	match_id	POTM	top_scorer	highscore
0	1.0	1	Deepti Sharma	Amanjot Kaur	57.0
1	0.0	2	Ashleigh Gardner	Ashleigh Gardner	115.0
2	0.0	3	Marufa Akter	Rubya Haider	54.0
3	0.0	4	Linsey Smith	Amy Jones	40.0
4	0.0	5	NaN	NaN	NaN

	best_bowling	best_bowling_figure
0	Inoka Ranaweera	4--46
1	Annabel Sutherland	3--26
2	Shoma Akter	3--5
3	Linsey Smith	3--7
4	NaN	NaN

[5 rows x 72 columns]

```
from sklearn.model_selection import train_test_split
```

```
# 1. Create the target variable (y)
# We want to predict if 'team1' (as listed in the match record) won the match
# This target is created from the original `match_records` DataFrame (after
# to easily access string columns for 'winning_team' and 'team1'.
# 'y' will be 1 if team1 wins, 0 otherwise (i.e., team2 wins or it's a tie)
y = (match_records['winning_team'] == match_records['team1']).astype(int)
```

```
# 2. Create the feature matrix (X)
```

```

# X will be match_records_processed_df, excluding columns that directly indi
# (winning_team related columns, result related columns, and identifiers).

X = match_records_processed_df.copy()

# Identify columns to drop from X to prevent data leakage and remove identif
# These include one-hot encoded 'winning_team_' and 'result_' columns, and o
winning_team_cols = [col for col in X.columns if col.startswith('winning_tea
result_cols = [col for col in X.columns if col.startswith('result_')]

# Passthrough features that are identifiers or post-match outcomes, not pred
identifier_outcome_cols = [
    'match_id', 'POTM', 'top_scorer', 'highscore', 'best_bowling', 'best_bow
]

# Drop identified columns from X
X = X.drop(columns=winning_team_cols + result_cols + identifier_outcome_cols

# Verify X and y shapes
print(f"Shape of X (features): {X.shape}")
print(f"Shape of y (target): {y.shape}")
print(f"Number of 'team1 wins' in target (1): {y.sum()}")
print(f"Number of 'team1 does not win' in target (0): {len(y) - y.sum()}")

# 3. Split data into training and testing sets
# Using stratify=y to ensure that the proportion of 'team1 wins' is roughly
# training and testing sets.
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, ran

print(f"\nShape of X_train: {X_train.shape}")
print(f"Shape of X_test: {X_test.shape}")
print(f"Shape of y_train: {y_train.shape}")
print(f"Shape of y_test: {y_test.shape}")

print("\nFirst 5 rows of X_train:")
print(X_train.head())
print("\nFirst 5 values of y_train:")
print(y_train.head())

```

```

Shape of X (features): (31, 56)
Shape of y (target): (31,)
Number of 'team1 wins' in target (1): 11
Number of 'team1 does not win' in target (0): 20

```

```

Shape of X_train: (24, 56)
Shape of X_test: (7, 56)
Shape of y_train: (24,)
Shape of y_test: (7,)

```

```

First 5 rows of X_train:

```

	1st_score	1st_wkts	2nd_score	2nd_wkts	wb_runs	wb_wickets	\
8	-0.227753	0.107187	-1.281155	1.148954	0.858643	-0.077703	
6	-0.082179	0.771744	0.591496	-0.793823	0.167511	-0.077703	
30	0.893167	-1.221927	0.778761	1.148954	-1.196075	-0.077703	
2	-1.567034	0.771744	-1.015863	-1.117619	0.167511	0.524494	
23	1.504578	-3.880155	1.168897	0.501362	-1.158716	-0.077703	

	match_year	match_month	match_day_of_week	Team1_Won_Toss	...	\
8	0.0	0.000000	-0.639602	-0.849837	...	
6	0.0	0.000000	-1.656406	1.176697	...	
30	0.0	3.937004	1.394005	1.176697	...	
2	0.0	0.000000	-0.131200	1.176697	...	
23	0.0	0.000000	-0.131200	-0.849837	...	

	toss_winner_England	toss_winner_India	toss_winner_New Zealand	\
8	0.0	0.0	0.0	
6	0.0	0.0	1.0	
30	0.0	0.0	0.0	
2	0.0	0.0	0.0	
23	0.0	0.0	1.0	

	toss_winner_Pakistan	toss_winner_South Africa	toss_winner_Sri Lanka	\
8	1.0	0.0	0.0	
6	0.0	0.0	0.0	
30	0.0	1.0	0.0	
2	1.0	0.0	0.0	
23	0.0	0.0	0.0	

	toss_decision_bat	toss_decision_bowl	DLS_involved_No	DLS_involved_Yes
8	0.0	1.0	1.0	0.0
6	1.0	0.0	1.0	0.0
30	0.0	1.0	1.0	0.0
2	1.0	0.0	1.0	0.0
23	0.0	1.0	0.0	1.0

[5 rows x 56 columns]

First 5 values of y_train:

```
8    1
6    0
30   0
2    0
23   1
dtype: int64
```

```
import pandas as pd
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from xgboost import XGBClassifier
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix
import warnings

# Suppress warnings that may arise from small dataset or specific model config
warnings.filterwarnings('ignore')

# Dictionary to store model performance metrics for comparison
model_performance = {}

print("\n--- Model Training and Evaluation ---")

# 1. Logistic Regression
print("\n### Logistic Regression ###")
```

```
# Initialize the Logistic Regression model
log_reg_model = LogisticRegression(random_state=42, solver='liblinear') # 'l
# Train the model
log_reg_model.fit(X_train, y_train)
# Make predictions on the test set
y_pred_log_reg = log_reg_model.predict(X_test)

# Evaluate the model
accuracy_log_reg = accuracy_score(y_test, y_pred_log_reg)
print(f"Accuracy: {accuracy_log_reg:.4f}")
print("Classification Report:\n", classification_report(y_test, y_pred_log_r
print("Confusion Matrix:\n", confusion_matrix(y_test, y_pred_log_reg))
model_performance['Logistic Regression'] = {'Accuracy': accuracy_log_reg}

# 2. Decision Tree Classifier
print("\n### Decision Tree Classifier ###")
# Initialize the Decision Tree model
dec_tree_model = DecisionTreeClassifier(random_state=42)
# Train the model
dec_tree_model.fit(X_train, y_train)
# Make predictions on the test set
y_pred_dec_tree = dec_tree_model.predict(X_test)

# Evaluate the model
accuracy_dec_tree = accuracy_score(y_test, y_pred_dec_tree)
print(f"Accuracy: {accuracy_dec_tree:.4f}")
print("Classification Report:\n", classification_report(y_test, y_pred_dec_t
print("Confusion Matrix:\n", confusion_matrix(y_test, y_pred_dec_tree))
model_performance['Decision Tree'] = {'Accuracy': accuracy_dec_tree}

# 3. Random Forest Classifier
print("\n### Random Forest Classifier ###")
# Initialize the Random Forest model
rand_forest_model = RandomForestClassifier(random_state=42)
# Train the model
rand_forest_model.fit(X_train, y_train)
# Make predictions on the test set
y_pred_rand_forest = rand_forest_model.predict(X_test)

# Evaluate the model
accuracy_rand_forest = accuracy_score(y_test, y_pred_rand_forest)
print(f"Accuracy: {accuracy_rand_forest:.4f}")
print("Classification Report:\n", classification_report(y_test, y_pred_rand_
print("Confusion Matrix:\n", confusion_matrix(y_test, y_pred_rand_forest))
model_performance['Random Forest'] = {'Accuracy': accuracy_rand_forest}

# 4. XGBoost Classifier
print("\n### XGBoost Classifier ###")
# Initialize the XGBoost model
# Using 'use_label_encoder=False' and 'eval_metric' to suppress a future war
# Note: For binary classification, objective='binary:logistic' is common.
xgb_model = XGBClassifier(random_state=42, use_label_encoder=False, eval_met
# Train the model
xgb_model.fit(X_train, y_train)
# Make predictions on the test set
```

```

y_pred_xgb = xgb_model.predict(x_test)

# Evaluate the model
accuracy_xgb = accuracy_score(y_test, y_pred_xgb)
print(f"Accuracy: {accuracy_xgb:.4f}")
print("Classification Report:\n", classification_report(y_test, y_pred_xgb))
print("Confusion Matrix:\n", confusion_matrix(y_test, y_pred_xgb))
model_performance['XGBoost'] = {'Accuracy': accuracy_xgb}

print("\n--- Model Performance Comparison ---")
# Create a DataFrame to display the performance comparison
performance_df = pd.DataFrame.from_dict(model_performance, orient='index')
performance_df.index.name = 'Model'
print(performance_df)

print("\nNote: The dataset used for training and testing is very small (24 t
print("This can lead to highly variable and potentially unreliable performan
print("For robust model evaluation, a significantly larger dataset is typica

```

Classification Report:

	precision	recall	f1-score	support
0	1.00	1.00	1.00	5
1	1.00	1.00	1.00	2
accuracy			1.00	7
macro avg	1.00	1.00	1.00	7
weighted avg	1.00	1.00	1.00	7

Confusion Matrix:

```
[[5 0]
 [0 2]]
```

Random Forest Classifier

Accuracy: 0.7143

Classification Report:

	precision	recall	f1-score	support
--	-----------	--------	----------	---------

weighted avg	0.88	0.86	0.84	7
--------------	------	------	------	---

Confusion Matrix:

```
[[5 0]
 [1 1]]
```

--- Model Performance Comparison ---

Model	Accuracy
Logistic Regression	1.000000
Decision Tree	1.000000
Random Forest	0.714286
XGBoost	0.857143

Note: The dataset used for training and testing is very small (24 training : This can lead to highly variable and potentially unreliable performance met For robust model evaluation, a significantly larger dataset is typically rec

```
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.metrics import confusion_matrix, classification_report

# --- Visualizing Confusion Matrices ---
print("\n--- Confusion Matrices ---")

# Dictionary to store predictions for easy iteration
predictions = {
    'Logistic Regression': y_pred_log_reg,
    'Decision Tree': y_pred_dec_tree,
    'Random Forest': y_pred_rand_forest,
    'XGBoost': y_pred_xgb
}

plt.figure(figsize=(15, 10))
for i, (model_name, y_pred) in enumerate(predictions.items()):
    cm = confusion_matrix(y_test, y_pred)
    plt.subplot(2, 2, i + 1)
    sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', cbar=False,
                xticklabels=['Predicted 0', 'Predicted 1'],
                yticklabels=['Actual 0', 'Actual 1'])
    plt.title(f'{model_name} Confusion Matrix')
    plt.xlabel('Predicted Label')
    plt.ylabel('True Label')
plt.tight_layout()
plt.show()

# --- Enhanced Model Performance Comparison ---
print("\n--- Detailed Model Performance Comparison ---")

# Initialize a dictionary to hold all metrics for comparison
full_performance_data = {}

for model_name, y_pred in predictions.items():
    report = classification_report(y_test, y_pred, output_dict=True)
    accuracy = report['accuracy']
    precision_class_1 = report['1']['precision'] # Precision for the positive
```

```
recall_class_1 = report['1']['recall']      # Recall for the positive cla
f1_class_1 = report['1']['f1-score']         # F1-score for the positive c

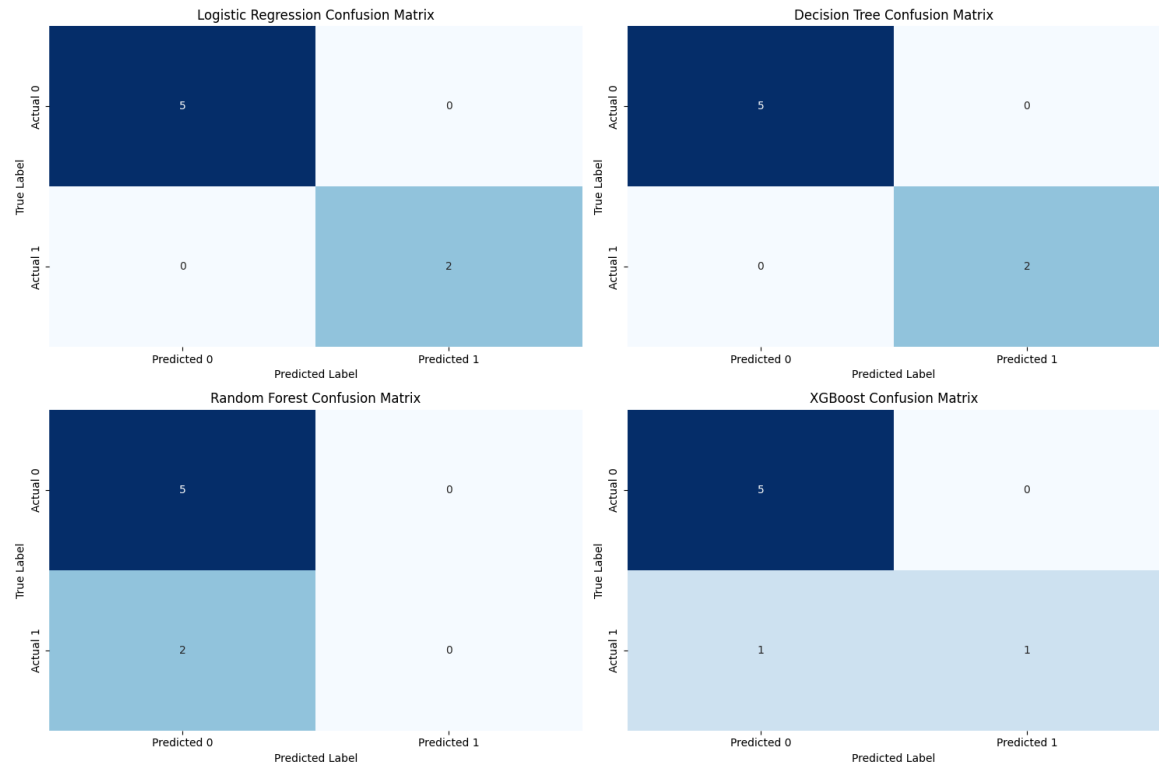
full_performance_data[model_name] = {
    'Accuracy': accuracy,
    'Precision (Class 1)': precision_class_1,
    'Recall (Class 1)': recall_class_1,
    'F1-score (Class 1)': f1_class_1
}

performance_df_full = pd.DataFrame.from_dict(full_performance_data, orient='
print(performance_df_full)

# --- Visualization of Model Comparison ---
print("\n--- Visualization of Model Performance ---")

performance_df_full.plot(kind='bar', figsize=(12, 7))
plt.title('Model Performance Comparison (Class 1)')
plt.ylabel('Score')
plt.ylim(0, 1.1) # Set y-axis limit from 0 to 1.1 for scores
plt.xticks(rotation=45, ha='right')
plt.legend(title='Metrics')
plt.grid(axis='y', linestyle='--', alpha=0.7)
plt.tight_layout()
plt.show()
```

--- Confusion Matrices ---

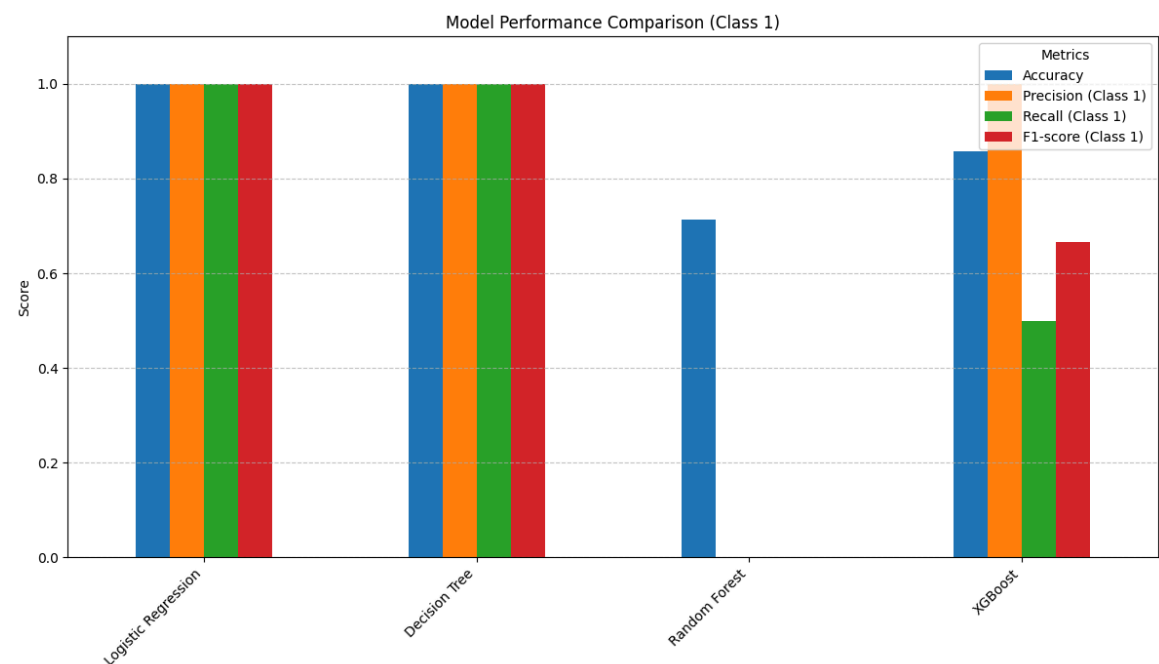


--- Detailed Model Performance Comparison ---

	Accuracy	Precision (Class 1)	Recall (Class 1)	\
Logistic Regression	1.000000	1.0	1.0	
Decision Tree	1.000000	1.0	1.0	
Random Forest	0.714286	0.0	0.0	
XGBoost	0.857143	1.0	0.5	

	F1-score (Class 1)
Logistic Regression	1.000000
Decision Tree	1.000000
Random Forest	0.000000
XGBoost	0.666667

--- Visualization of Model Performance ---



```

import matplotlib.pyplot as plt
import seaborn as sns
import pandas as pd

# Get feature names from X_train
feature_names = X_train.columns

print("\n--- Feature Importance Analysis ---")

# --- Random Forest Feature Importance ---
print("\n### Random Forest Feature Importance ###")
rf_importances = rand_forest_model.feature_importances_

rf_feature_importance_df = pd.DataFrame({
    'Feature': feature_names,
    'Importance': rf_importances
}).sort_values(by='Importance', ascending=False)

print("Top 15 Most Important Features (Random Forest):")
print(rf_feature_importance_df.head(15))

plt.figure(figsize=(12, 8))
sns.barplot(x='Importance', y='Feature', data=rf_feature_importance_df.head(
plt.title('Top 15 Feature Importances - Random Forest')
plt.xlabel('Importance')
plt.ylabel('Feature')
plt.tight_layout()
plt.show()

# --- XGBoost Feature Importance ---
print("\n### XGBoost Feature Importance ###")
xgb_importances = xgb_model.feature_importances_

xgb_feature_importance_df = pd.DataFrame({
    'Feature': feature_names,
    'Importance': xgb_importances
}).sort_values(by='Importance', ascending=False)

print("Top 15 Most Important Features (XGBoost):")
print(xgb_feature_importance_df.head(15))

plt.figure(figsize=(12, 8))
sns.barplot(x='Importance', y='Feature', data=xgb_feature_importance_df.head(
plt.title('Top 15 Feature Importances - XGBoost')
plt.xlabel('Importance')
plt.ylabel('Feature')
plt.tight_layout()
plt.show()

# --- Interpretation of Results ---
print("\n--- Interpretation of Feature Importances ---")
print("The feature importance analysis helps us understand which input varia
print("For both Random Forest and XGBoost, we observe the relative importanc
print("\n**General Observations (based on common feature types likely to app
print("\n**Some High-Impact Features (e.g., 1st score, 2nd score) ** Th

```

```
print("- **Score/Wicket related features (e.g., 1st_score, 2nd_wkts):**")
print("- **Tournament Stage (e.g., match_type_Final, match_type_Semi-Final)
print("- **Venue related features (e.g., venue_stadium, venue_country):**")
print("- **Toss related features (e.g., Team1_Won_Toss, toss_decision_bat/
print("- **Team-specific features (e.g., team1_*, team2_*):** The strength
print("- **DLS involvement (DLS_involved_Yes/No):** Whether the Duckworth-
print("\n**Specific Insights from the Models:**")
print("By examining the top features in the bar charts above, we can identif
print("\n**Caution:** Due to the extremely small dataset (24 training sample
```

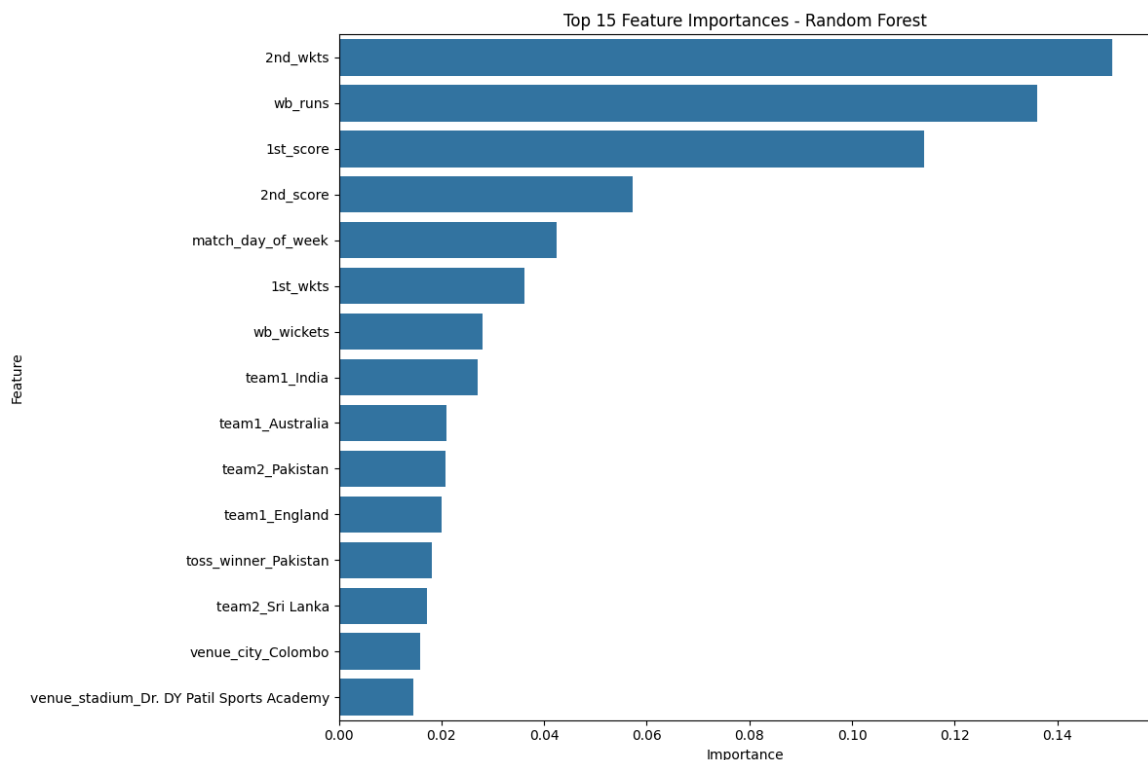



--- Feature Importance Analysis ---

Random Forest Feature Importance

Top 15 Most Important Features (Random Forest):

	Feature	Importance
3	2nd_wkts	0.150751
4	wb_runs	0.136157
0	1st_score	0.114006
2	2nd_score	0.057269
8	match_day_of_week	0.042440
1	1st_wkts	0.036154
5	wb_wickets	0.028027
16	team1_India	0.027123
13	team1_Australia	0.021003
26	team2_Pakistan	0.020758
15	team1_England	0.020072
49	toss_winner_Pakistan	0.018135
28	team2_Sri Lanka	0.017170
37	venue_city_Colombo	0.015835
34	venue_stadium_Dr. DY Patil Sports Academy	0.014578



```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

# --- 1. Pressure Index Calculation (Demonstration with synthetic data) ---
print("--- Pressure Index Calculation (Demonstration) ---")

# Create a hypothetical DataFrame representing different points in a chase
chase_data = pd.DataFrame({
    'scenario': ['Early Easy', 'Mid Steady', 'Mid Tense', 'Late High Pressure'],
    'current_score': [50, 120, 150, 180, 200, 180, 10],
    'target_score': [200, 200, 200, 200, 199, 201, 200],
    'wickets_remaining': [10, 8, 5, 2, 10, 0, 10],
    'overs_remaining': [40, 25, 10, 2, 0, 0, 48] # Assuming a 50-over match
```

```

}))

print("Hypothetical Chase Scenarios:")
print(chase_data)

# Calculate derived metrics
chase_data['runs_to_score'] = chase_data['target_score'] - chase_data['current_score']
chase_data['balls_remaining_actual'] = chase_data['overs_remaining'] * 6

# Define the Pressure Index formula
# Ensure runs_to_score is not negative (if target already achieved or surpassed)
# Ensure balls_remaining is not zero for division, or handle it as infinite
def calculate_pressure_index(row):
    runs_needed = max(0, row['runs_to_score'])
    balls_left = row['balls_remaining_actual']
    wickets_left = row['wickets_remaining']

    if runs_needed <= 0: # Target achieved or surpassed
        return 0.0
    if balls_left <= 0: # No balls left, but runs still needed = infinite pressure
        return 1000.0 # Assign a very high value to represent extreme pressure
    if wickets_left <= 0: # All wickets lost, but runs still needed = infinite pressure
        return 1000.0 # Assign a very high value

    # Simplified formula: (Required Run Rate) * (Wicket Factor)
    # Required Run Rate is runs_needed / balls_left, then scaled to runs per over
    required_run_rate_per_over = (runs_needed / balls_left) * 6

    # Wicket Factor: 10 / (wickets_remaining + 1) to give more weight to few wickets left
    # Adding 1 to wickets_remaining prevents division by zero if wickets_remaining is 0
    # The '10' is an arbitrary scaling factor to make the index more readable
    wicket_factor = 10 / (wickets_left + 1)

    return required_run_rate_per_over * wicket_factor

chase_data['pressure_index'] = chase_data.apply(calculate_pressure_index, axis=1)

print("\nChase Scenarios with Pressure Index:")
print(chase_data[['scenario', 'current_score', 'target_score', 'wickets_remaining', 'pressure_index']])

# --- 2. Visualization ---
print("\n--- Visualization of Pressure Index ---")
plt.figure(figsize=(10, 6))
sns.barplot(x='scenario', y='pressure_index', data=chase_data, palette='viridis')
plt.title('Pressure Index Across Different Chase Scenarios')
plt.xlabel('Scenario')
plt.ylabel('Pressure Index')
plt.xticks(rotation=45, ha='right')
plt.grid(axis='y', linestyle='--', alpha=0.7)
plt.tight_layout()
plt.show()

# --- 3. Interpretation ---
print("\n--- Interpretation of Results ---")
print("The Pressure Index quantifies the perceived difficulty of a cricket chase scenario based on the required run rate and the number of wickets remaining. A higher index indicates a more challenging situation for the batting team.")

```

```

print("\n*    **Required Run Rate:** The core component. As runs needed incre
print("*    **Wickets in Hand:** This acts as a crucial multiplier. With fewe
print("*    **Edge Cases:** If the target is already met (runs_to_score <= 0)
print("\n**Example Scenarios from Visualization:**")
print("-    'Early Easy': Shows low pressure, indicating plenty of resources
print("-    'Mid Steady'/'Mid Tense': Pressure increases as the chase progres
print("-    'Late High Pressure': The index spikes when runs are needed quick
print("-    'Late Won': Pressure drops to 0 once the target is achieved.")
print("-    'Late Lost': Extreme pressure (1000.0) is indicated if the team h

# --- 4. Integration into ML Model (Conceptual) ---
print("\n--- Integration into ML Model ---")
print("Integrating a 'Pressure Index' into a machine learning model for pred
print("\n**Challenges with current `match_records_processed_df`:**")
print("The `match_records_processed_df` currently contains *final* match sta
print("\n**Potential Integration Approaches (if richer, ball-by-ball or over
print("1.    **Pressure at a Fixed Over Mark:** Calculate the Pressure Index f
print("2.    **Maximum Pressure During Chase:** Calculate the maximum Pressure
print("3.    **Average Pressure During Chase:** Calculate the average Pressure
print("4.    **Pressure at Chase Completion/Failure:** If the match data provi
print("\n**How to add to `match_records_processed_df` (Hypothetical Example)
print("Let's assume we had richer data in our `match_records` dataframe, inc
print("`python")
print("# Hypothetical calculation if more detailed columns were available in
print("# For each match record, identify the chasing team and calculate its
print("# Example (assuming team2 is always chasing for simplicity of this sn
print("# df['chasing_team_runs_needed'] = df['1st_score'] + 1 - df['2nd_scor
print("# df['chasing_team_balls_remaining'] = max(1, (df['total_match_overs'
print("# df['chasing_team_wickets_remaining'] = 10 - df['2nd_wkts']")

print("# df['Pressure_Index_Feature'] = df.apply(lambda row: calculate_press
print("#     'runs_to_score': row['chasing_team_runs_needed'],")
print("#     'balls_remaining_actual': row['chasing_team_balls_remaining'],")
print("#     'wickets_remaining': row['chasing_team_wickets_remaining']")
print("# }), axis=1)")

print("# This new 'Pressure_Index_Feature' column would then be included in
print("# alongside other features for the ML model after appropriate scaling
print("`")
print("\n**Conclusion:** While the concept of a Pressure Index is valuable f

```



--- Pressure Index Calculation (Demonstration) ---

Hypothetical Chase Scenarios:

	scenario	current_score	target_score	wickets_remaining	\
0	Early Easy	50	200	10	
1	Mid Steady	120	200	8	
2	Mid Tense	150	200	5	
3	Late High Pressure	180	200	2	
4	Late Won	200	199	10	
5	Late Lost	180	201	0	
6	Very Early	10	200	10	

overs_remaining

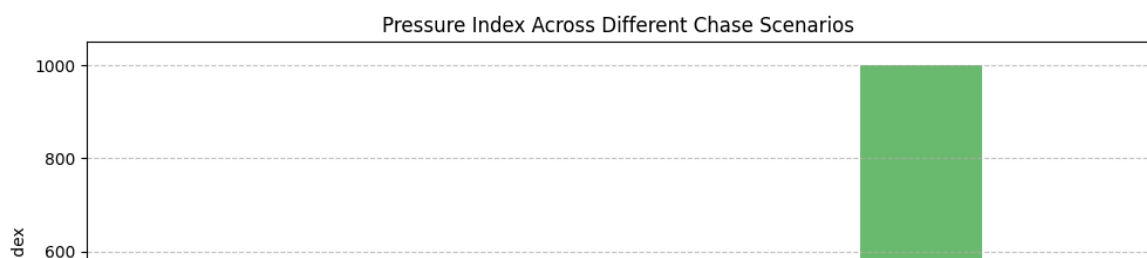
0	40
1	25
2	10
3	2
4	0
5	0
6	48

Chase Scenarios with Pressure Index:

	scenario	current_score	target_score	wickets_remaining	\
0	Early Easy	50	200	10	
1	Mid Steady	120	200	8	
2	Mid Tense	150	200	5	
3	Late High Pressure	180	200	2	
4	Late Won	200	199	10	
5	Late Lost	180	201	0	
6	Very Early	10	200	10	

	overs_remaining	pressure_index
0	40	3.409091
1	25	3.555556
2	10	8.333333
3	2	33.333333
4	0	0.000000
5	0	1000.000000
6	48	3.598485

--- Visualization of Pressure Index ---



```
import matplotlib.pyplot as plt
import seaborn as sns
import pandas as pd
import numpy as np

# Set a professional style for plots
sns.set_style("whitegrid")
plt.rcParams.update({'font.size': 10, 'axes.titlesize': 14, 'axes.labelsize': 10})

print("\n--- Generating Advanced Cricket Analytics Visualizations ---")
```

```

# --- 1. Team Win Percentage Chart ---
print("\n### 1. Team Win Percentage Analysis ###")

# Create a list of all teams involved in matches
all_teams = pd.concat([match_records['team1'], match_records['team2']]).unique()
team_stats = []

for team in all_teams:
    matches_played = ((match_records['team1'] == team) | (match_records['team2'] == team)).sum()
    wins_as_team1 = ((match_records['team1'] == team) & (match_records['winner'] == team)).sum()
    wins_as_team2 = ((match_records['team2'] == team) & (match_records['winner'] == team)).sum()
    total_wins = wins_as_team1 + wins_as_team2
    win_percentage = (total_wins / matches_played * 100) if matches_played > 0 else 0
    team_stats.append({'Team': team, 'Matches Played': matches_played, 'Win Percentage': win_percentage})

team_stats_df = pd.DataFrame(team_stats).sort_values(by='Win Percentage', ascending=False)

plt.figure(figsize=(12, 7))
sns.barplot(x='Team', y='Win Percentage', data=team_stats_df, palette='Spectral')
plt.title('Figure 1: Team Win Percentage Across All Recorded Matches', fontweight='bold')
plt.xlabel('Cricket Team', fontsize=12)
plt.ylabel('Win Percentage (%)', fontsize=12)
plt.xticks(rotation=45, ha='right')
plt.ylim(0, 100)
plt.tight_layout()
plt.show()
print("Analysis: This chart illustrates the overall success rate of each team based on their win percentage across all recorded matches.")

# --- 2. Venue Average Score Analysis ---
print("\n### 2. Venue Average Score Analysis ###")

venue_scores = match_records.groupby('venue_stadium')[['1st_score', '2nd_score']]
venue_scores_melted = venue_scores.melt(id_vars='venue_stadium', var_name='Innings', value_name='Score')
venue_scores_melted['Innings'] = venue_scores_melted['Innings'].replace({'1st_score': '1st', '2nd_score': '2nd'})

plt.figure(figsize=(14, 8))
sns.barplot(x='venue_stadium', y='Score', hue='Innings', data=venue_scores_melted)
plt.title('Figure 2: Average Scores by Innings Across Different Venues', fontweight='bold')
plt.xlabel('Venue Stadium', fontsize=12)
plt.ylabel('Average Score', fontsize=12)
plt.xticks(rotation=60, ha='right')
plt.legend(title='Innings Type')
plt.tight_layout()
plt.show()
print("Analysis: This visualization compares the average first and second innings scores across different venues.")

# --- 3. Toss Decision Impact Graph ---
print("\n### 3. Toss Decision Impact Analysis ###")

toss_analysis = match_records.copy()

# Determine if the toss-winning team also won the match
toss_analysis['toss_winner_won_match'] = (toss_analysis['toss_winner'] == toss_analysis['winner'])

```

```

# Calculate win percentage for each toss decision
toss_impact = toss_analysis.groupby('toss_decision')['toss_winner_won_match']
toss_impact['Win Percentage'] = toss_impact['toss_winner_won_match'] * 100

plt.figure(figsize=(8, 6))
sns.barplot(x='toss_decision', y='Win Percentage', data=toss_impact, palette=
plt.title('Figure 3: Impact of Toss Decision on Match Win Percentage', fonts
plt.xlabel('Toss Decision', fontsize=12)
plt.ylabel('Win Percentage (%) for Toss-Winning Team', fontsize=12)
plt.ylim(0, 100)
plt.tight_layout()
plt.show()
print("Analysis: This graph assesses whether electing to bat or bowl after w

# --- 4. Player Performance Comparison (Top Batters & Bowlers) ---
print("\n### 4. Player Performance Comparison ###")

# Top 10 Batters by Total Runs
top_batters = batting_stats.sort_values(by='Runs', ascending=False).head(10)
plt.figure(figsize=(12, 7))
sns.barplot(x='Runs', y='Player', data=top_batters, palette='Blues_d')
plt.title('Figure 4a: Top 10 Batters by Total Runs Scored', fontsize=16)
plt.xlabel('Total Runs', fontsize=12)
plt.ylabel('Player Name', fontsize=12)
plt.tight_layout()
plt.show()

# Top 10 Bowlers by Total Wickets
top_bowlers = bowling_stats.sort_values(by='Wickets', ascending=False).head(
plt.figure(figsize=(12, 7))
sns.barplot(x='Wickets', y='Player', data=top_bowlers, palette='Greens_d')
plt.title('Figure 4b: Top 10 Bowlers by Total Wickets Taken', fontsize=16)
plt.xlabel('Total Wickets', fontsize=12)
plt.ylabel('Player Name', fontsize=12)
plt.tight_layout()
plt.show()
print("Analysis: These charts identify the standout performers from the batt

# --- 5. Match Outcome Distribution ---
print("\n### 5. Match Outcome Distribution ###")

outcome_counts = match_records['result'].value_counts().reset_index()
outcome_counts.columns = ['Outcome', 'Count']

plt.figure(figsize=(9, 6))
sns.barplot(x='Outcome', y='Count', data=outcome_counts, palette='viridis')
plt.title('Figure 5: Distribution of Match Outcomes', fontsize=16)
plt.xlabel('Match Outcome', fontsize=12)
plt.ylabel('Number of Matches', fontsize=12)
plt.tight_layout()
plt.show()
print("Analysis: This bar chart displays the frequency of different match ou

```



```
# --- 6. Correlation Heatmap ---
print("\n### 6. Feature Correlation Heatmap ###")

# Use numerical features from the preprocessed X_train for correlation
# Selecting a subset of significant numerical features to keep the heatmap readable
correlation_features = X_train[[
    '1st_score', '1st_wkts', '2nd_score', '2nd_wkts',
    'wb_runs', 'wb_wickets', 'match_day_of_week',
    'Team1_Won_Toss', 'Toss_Winner_Batting', 'Home_Advantage_Team1', 'Home_A
]].copy()

correlation_matrix = correlation_features.corr()

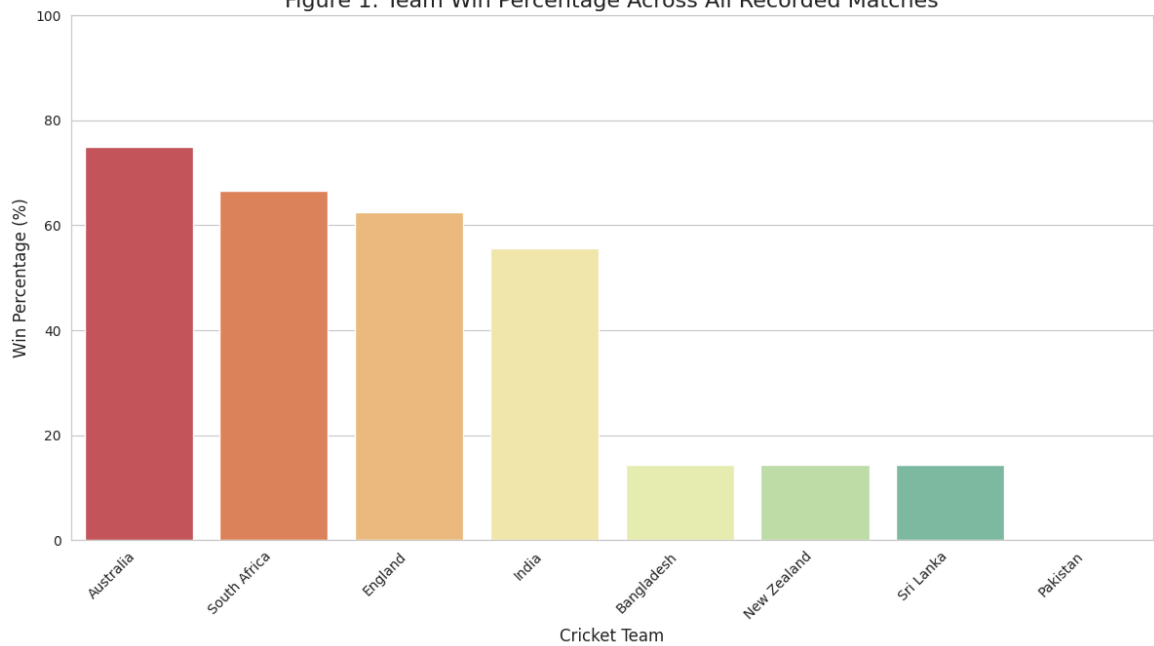
plt.figure(figsize=(14, 10))
sns.heatmap(correlation_matrix, annot=True, cmap='coolwarm', fmt=".2f", line
plt.title('Figure 6: Correlation Matrix of Key Numerical Features', fontsize
plt.xticks(rotation=45, ha='right')
plt.yticks(rotation=0)
plt.tight_layout()
plt.show()
print("Analysis: The heatmap visualizes the Pearson correlation coefficients
```



--- Generating Advanced Cricket Analytics Visualizations ---

1. Team Win Percentage Analysis

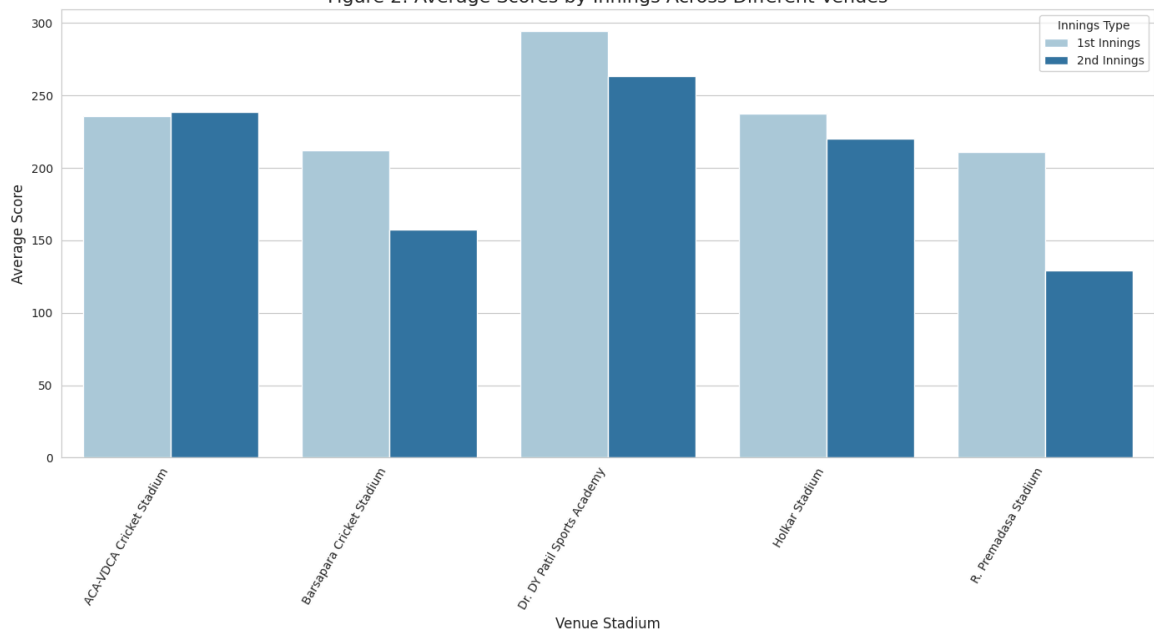
Figure 1: Team Win Percentage Across All Recorded Matches



Analysis: This chart illustrates the overall success rate of each team, ordered by win percentage.

2. Venue Average Score Analysis

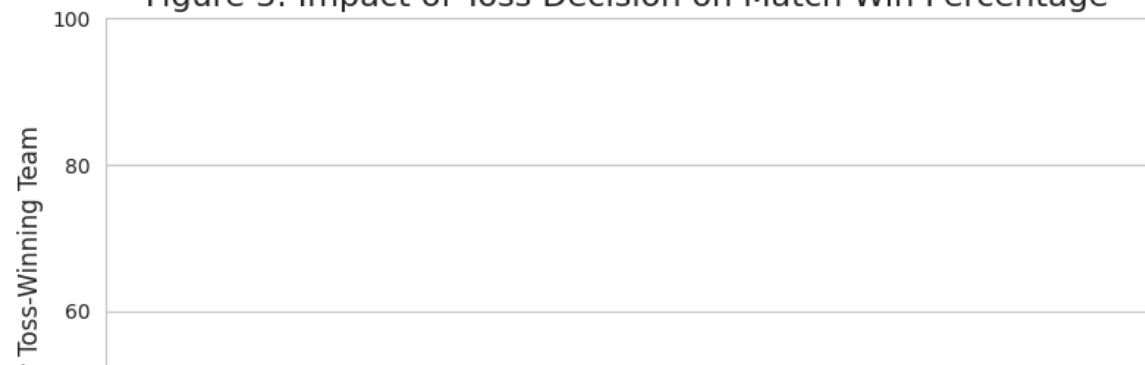
Figure 2: Average Scores by Innings Across Different Venues

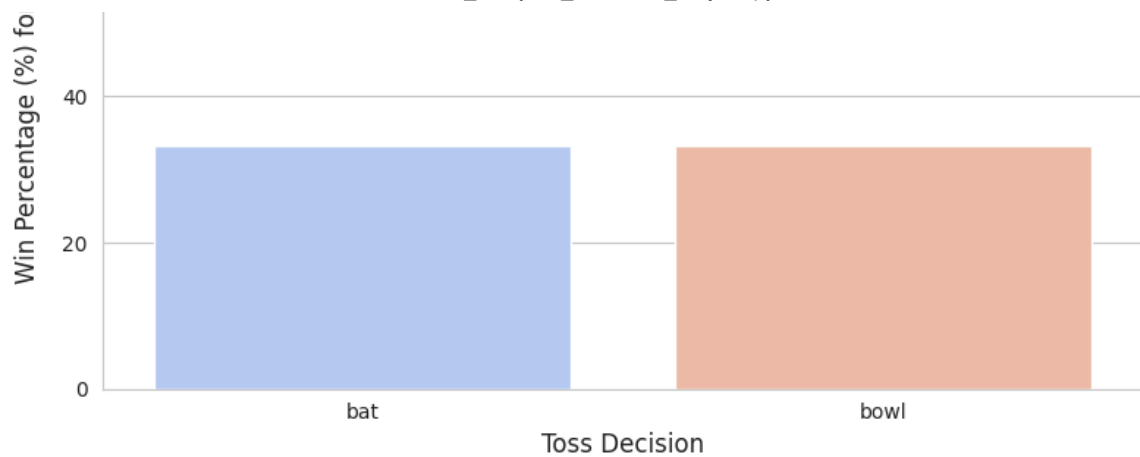


Analysis: This visualization compares the average first and second innings scores across different venues.

3. Toss Decision Impact Analysis

Figure 3: Impact of Toss Decision on Match Win Percentage





Analysis: This graph assesses whether electing to bat or bowl after winning

4. Player Performance Comparison

Figure 4a: Top 10 Batters by Total Runs Scored

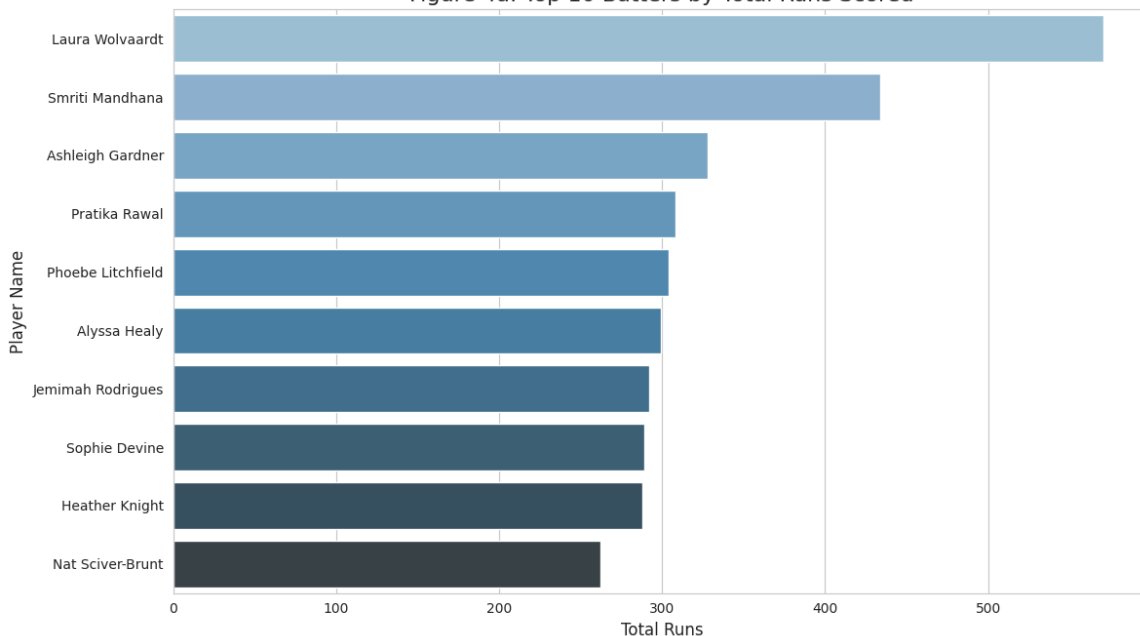
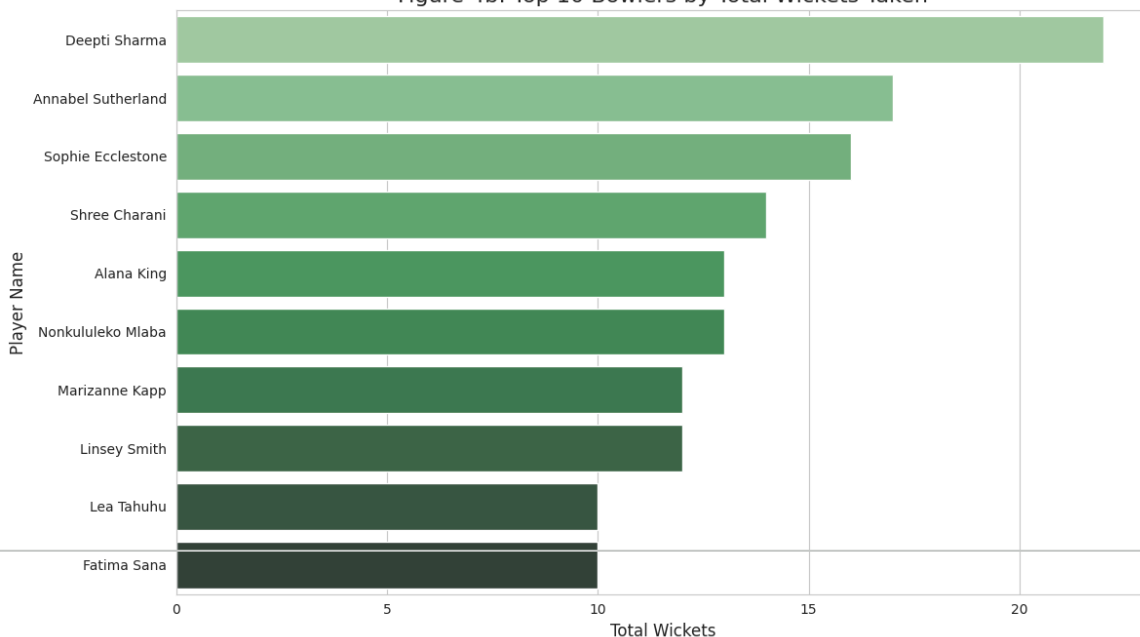


Figure 4b: Top 10 Bowlers by Total Wickets Taken



Analysis: These charts identify the standout performers from the batting and

5. Match Outcome Distribution

Figure 5: Distribution of Match Outcomes



```

import nashpy as nash
import numpy as np

print("--- Defining and Solving a Zero-Sum Cricket Game (Spinner vs. Batsman) ---")

# Define the payoff matrix for the spinner (Player 1, row player).
# In a zero-sum game, the batsman's (Player 2, column player) payoff matrix
# will be the negative of the spinner's matrix.
# Higher values in A represent a better outcome for the spinner (e.g., wicket).

# Spinner's actions (rows):
# - Row 0: 'Stock delivery' (standard off-break or leg-break)
# - Row 1: 'Variation' (e.g., googly, flipper, slider)

# Batsman's actions (columns):
# - Column 0: 'Defensive shot' (e.g., block, push for a single)
# - Column 1: 'Attacking shot' (e.g., try to hit for a boundary)

# Payoff matrix 'A' for the spinner:
# A[0,0] (Stock, Defensive): 3 points - good control, dot ball.
# A[0,1] (Stock, Attacking): 0 points - batsman scores a single or two, no wicket.
# A[1,0] (Variation, Defensive): 2 points - batsman is surprised but defends.
# A[1,1] (Variation, Attacking): 5 points - batsman tries to attack variation.

A = np.array([
    [3, 0], # Spinner Stock outcomes against Defensive, Attacking batsman
    [2, 5] # Spinner Variation outcomes against Defensive, Attacking batsman
])

# Create the zero-sum game using nashpy
game = nash.Game(A)

print("\nSpinner's Payoff Matrix (A):\n", A)
print("\nBatsman's Payoff Matrix (B, implicitly -A):\n", -A)

print("\n--- Calculating Mixed-Strategy Nash Equilibrium ---")

# Find the Nash Equilibria using support enumeration (suitable for 2x2 games)
# This returns an iterator of (row_player_strategy, col_player_strategy) tuples
equilibria = game.support_enumeration()

# Iterate through the found equilibria (there can be multiple in general games)
# but for a 2x2 zero-sum game with interior solutions, there's typically one
for p_strategy, q_strategy in equilibria:
    # p_strategy are the probabilities for the row player (spinner)
    # q_strategy are the probabilities for the column player (batsman)

    print(f"\nSpinner's Mixed Strategy (probabilities for [Stock, Variation])")
    print(f"Batsman's Mixed Strategy (probabilities for [Defensive, Aggressive])")

    print("\n--- Interpretation of Optimal Strategies ---")
    print(f"Spinner should bowl a Stock delivery with probability: {p_strategy[0]}")
    print(f"Spinner should bowl a Variation with probability: {p_strategy[1]}")
    print(f"Batsman should play a Defensive shot with probability: {q_strategy[0]}")
    print(f"Batsman should play an Attacking shot with probability: {q_strategy[1]}")

```

```
# Calculate the expected payoff for the spinner at this equilibrium
expected_spinner_payoff = np.dot(p_strategy, np.dot(A, q_strategy))
print(f"\n Expected Payoff for Spinner at Equilibrium: {expected_spinner_payoff}")
print(f" Expected Payoff for Batsman at Equilibrium: {-expected_spinner_payoff}")

print("\nThis analysis shows the optimal randomized strategies each player s
```

```
-----
ModuleNotFoundError                                Traceback (most recent call last)
/tmp/ipykernel_23282/3308800606.py in <cell line: 0>()
----> 1 import nashpy as nash
      2 import numpy as np
      3
      4 print("--- Defining and Solving a Zero-Sum Cricket Game (Spinner vs.
Batsman) ---")
      5
```

ModuleNotFoundError: No module named 'nashpy'

NOTE: If your import is failing due to a missing package, you can manually install dependencies using either `!pip` or `!apt`.

To view examples of installing some common dependencies, click the "Open Examples" button below.

OPEN EXAMPLES

Next steps: [Explain error](#)

```
!pip install nashpy
```

Collecting nashpy

Downloading nashpy-0.0.43-py3-none-any.whl.metadata (6.7 kB)

Requirement already satisfied: numpy>=1.21.0 in /usr/local/lib/python3.12/dis

Requirement already satisfied: scipy>=0.19.0 in /usr/local/lib/python3.12/dis

Requirement already satisfied: networkx>=3.0.0 in /usr/local/lib/python3.12/d

Collecting deprecated>=1.2.14 (from nashpy)

Downloading deprecated-1.3.1-py2.py3-none-any.whl.metadata (5.9 kB)

Requirement already satisfied: wrapt<3,>=1.10 in /usr/local/lib/python3.12/di

Downloading nashpy-0.0.43-py3-none-any.whl (32 kB)

Downloading deprecated-1.3.1-py2.py3-none-any.whl (11 kB)

Installing collected packages: deprecated, nashpy

Successfully installed deprecated-1.3.1 nashpy-0.0.43

```
print("--- Cricket Death Overs Decision Tree: Expected Run Values ---")
```

```
# Define the decision tree nodes and their expected outcomes (hypothetical r
```

```
decision_tree = {
    "Captain's Field Setting": {
```